



Quiz 5-6

Name: Abdullah Shakeel

Class: BSSE

Roll no: 019

Submitted To: Miss Jawaria Ramzan

Subject: Software Verification & Validation

Section A – Validation Criteria and Checklist Development

Q1. Validation Criteria Engineering

1. Measurable Validation Criteria

System Feature	Stakeholder Expectation	Validation Criteria	Measurement Method
Student Login	Secure and fast authentication	Login success rate $\geq$ 95% and response time $\leq$ 3 seconds	Functional testing and response monitoring
Assignment Submission	Reliable file upload	Files up to 50 MB uploaded successfully without corruption	Upload validation test
Attendance Tracking	Accurate attendance records	Attendance accuracy = 100% compared with manual records	Data verification testing
Faculty Dashboard	Correct academic data display	Dashboard loads within 5 seconds and displays correct student data	UI and database validation
Activity Logging	Complete audit trail generation	All user activities logged with timestamp and user ID	Log inspection and audit testing

2. Structured Validation Checklist

University Management System Validation Checklist

Validation ID	Module	Validation Item	Expected Result	Validation Method	Status	Evidence
VAL-001	Login	Student enters valid credentials	Successful login	Functional Testing	Pass/Fail	Screenshot/Logs
VAL-002	Login	Invalid password handling	Error message displayed	Security Testing	Pass/Fail	Screenshot
VAL-003	Assignment	Upload PDF assignment	File uploaded successfully	Functional Testing	Pass/Fail	Upload Log
VAL-004	Attendance	Mark attendance	Attendance stored correctly	Database Validation	Pass/Fail	DB Record
VAL-005	Dashboard	Faculty dashboard loading	Dashboard opens within 5 sec	Performance Testing	Pass/Fail	Performance Log

VAL-006	Logging	User activity tracking	Logs generated with timestamp	Audit Validation	Pass/Fail	Log File
---------	---------	------------------------	-------------------------------	------------------	-----------	----------

Section B – Continuous Validation Automation using CI Pipelines

Q2. Validation Automation using GitHub Actions

1. Role of CI Pipelines in Continuous Validation

Continuous Integration (CI) pipelines automate software validation whenever developers push code to the repository.

Importance:

- Detects defects early
- Ensures software quality continuously
- Automates testing and validation
- Generates validation evidence automatically
- Improves deployment reliability
- Maintains audit and compliance records

CI pipelines help teams validate functionality, security, and performance consistently without manual intervention.

2. GitHub Actions Workflow

Code is pushed in [GitHub](#)

Workflow Explanation

Step	Purpose
Checkout Repository	Downloads project code
Setup Python	Configures runtime environment
Install Dependencies	Installs required libraries
Execute Tests	Runs automated validation tests
Generate Logs	Creates validation evidence
Upload Artifacts	Stores logs for audit/review

Section C – Security Validation and Vulnerability Assessment

Q3. Structural Vulnerability Examination using OWASP ZAP

1. Importance of Security-Oriented Validation

Security-oriented validation ensures that software systems are protected against cyber threats, unauthorized access, and data breaches before deployment.

Benefits:

- Prevents exploitation of vulnerabilities
- Protects confidential data
- Ensures regulatory compliance
- Reduces financial and reputational risks

- Improves software trustworthiness

2. Three Common Vulnerabilities

1. SQL Injection

Explanation

SQL Injection happens when an attacker enters harmful SQL code into input fields like login forms or search boxes. The application runs this code on the database.

Example

An attacker may type special SQL commands in the username or password field to log in without permission.

Impact

- Unauthorized access to the database
- Stealing or deleting data
- Changing database information

Prevention

- Validate user input
- Use prepared statements
- Restrict database permissions

2. Cross-Site Scripting (XSS)

Explanation

XSS happens when attackers add harmful scripts into a website. These scripts run in other users' browsers.

Example

An attacker inserts a JavaScript code into a comment or form field.

Impact

- Stealing user cookies or sessions
- Redirecting users to fake websites
- Running unwanted actions in the browser

Prevention

- Filter and sanitize user input
- Encode output properly
- Use secure browser settings

3. Broken Authentication

Explanation

Broken Authentication happens when login or session systems are weak and attackers can access user accounts.

Example

Using weak passwords or not logging users out properly.

Impact

- Unauthorized account access
- Account hacking
- Data theft

Prevention

- Use strong passwords
- Enable multi-factor authentication (MFA)
- Secure session management
- Limit failed login attempts

3. OWASP ZAP Vulnerability Assessment Workflow

OWASP ZAP Validation Workflow

Step 1 — Configure Target Application

- Install [OWASP ZAP Official Website](https://owasp.org/zap/)
- Launch ZAP
- Enter target URL: <https://demo.owasp-juice.shop/>

Step 2 — Passive Scanning

Purpose: Analyze HTTP traffic without attacking the application.

Activities:

- Header inspection
- Cookie analysis
- Session validation
- Security configuration checking

Output: Passive scan alerts generated automatically.

Step 3 — Active Scanning

Purpose:

Actively test vulnerabilities by sending attack payloads.

Activities:

- SQL Injection testing
- XSS testing
- Authentication testing

Process:

1. Right-click target
2. Select:

Attack → Active Scan

Step 4 — Risk Classification

OWASP ZAP classifies vulnerabilities into:

Risk Level	Meaning
High	Critical vulnerability
Medium	Serious security weakness
Low	Minor issue
Informational	Observation only

Step 5 — Validation Evidence Generation

Generate Reports:

1. Open:

Report → Generate HTML Report

2. Export:

- HTML report
- XML report
- JSON report

Evidence Includes:

- Vulnerability screenshots
- Alert logs
- Risk severity table

- Scan summary

Example Vulnerability Severity Table

Vulnerability	Risk Level	Affected Module	Recommendation
SQL Injection	High	Login Form	Use parameterized queries
XSS	Medium	Comment Section	Sanitize user input
Missing Security Headers	Low	HTTP Response	Add secure headers

Mitigation Recommendations

Vulnerability	Mitigation
SQL Injection	Prepared statements and ORM
XSS	Input sanitization and output encoding
Broken Authentication	MFA and secure session management
Security Misconfiguration	Secure default settings
Sensitive Data Exposure	HTTPS and encryption

Section D – Validation Risk and Compliance Documentation

Q4. Validation Risk Management and Compliance Evidence

1. Validation Risk Register

Risk ID	Risk Description	Impact	Probability	Mitigation Strategy
RISK-001	Unauthorized access to financial data	High	Medium	Implement MFA and RBAC
RISK-002	Sensitive customer data exposure	High	Medium	Encrypt data and use HTTPS
RISK-003	Missing audit logs	Medium	Low	Enable centralized logging
RISK-004	Non-compliance with regulations	High	Medium	Conduct regular compliance audits

2. Compliance Evidence Repository Structure

Structure seen in [GitHub repository](#)

Abdullah-shakee / Quiz-5-6

Search Type [f] to search

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Quiz-5-6

Public

Pin

Watch 0

main

1 Branch

0 Tags

Go to file

Add file

<> Code

Abdullah-shakee

Delete Validation report/Quiz report

67775a1 · now

10 Commits

github/workflow	add CI pipeline	39 minutes ago
Validation report	Delete Validation report/Quiz report	now
zap report	Add files via upload	14 minutes ago
Scan_Screenshot_1.png	Add files via upload	12 minutes ago
Scan_Screenshot_2.png	Add files via upload	11 minutes ago

README